

Tarea Semana 0: Mi Primer Programa en Python

Funciones, Variables y Tipos de Datos

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido al mundo de la programación en Python! Esta es tu primera tarea práctica del curso. El objetivo es que te familiarices con los conceptos más fundamentales de la programación: variables, funciones básicas, tipos de datos y la interacción con el usuario.

No te preocupes si al principio parece complicado. La programación es como aprender un nuevo idioma: con práctica y paciencia, todo cobra sentido. Esta tarea está diseñada para que aprendas haciendo, paso a paso.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Crear y ejecutar programas básicos en Python
- Utilizar las funciones `print()` e `input()` para interactuar con el usuario
- Declarar y usar variables con nombres descriptivos
- Trabajar con diferentes tipos de datos (strings, integers, floats)
- Realizar conversiones entre tipos de datos (type casting)
- Aplicar operadores aritméticos básicos
- Usar formateo de strings con f-strings
- Aplicar métodos básicos de strings
- Documentar tu código con comentarios claros

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear un programa interactivo llamado “**Calculadora de Propinas Personalizada**”. Este programa ayudará a calcular cuánto debe pagar cada persona cuando se divide una cuenta en un restaurante, incluyendo la propina.

El programa debe ser amigable, saludar al usuario por su nombre, y guiarlo paso a paso para obtener la información necesaria y mostrar los resultados de forma clara y profesional.

3.2. Requisitos Funcionales

Tu programa debe realizar las siguientes acciones en orden:

1. Saludo personalizado:

- Solicitar el nombre del usuario
- Dar la bienvenida usando el nombre (debe mostrarse con la primera letra en mayúscula)

2. Recopilación de datos:

- Preguntar el monto total de la cuenta del restaurante
- Preguntar el porcentaje de propina que desea dejar (ejemplo: 10, 15, 20)
- Preguntar cuántas personas van a dividir la cuenta

3. Cálculos:

- Calcular el monto de la propina
- Calcular el total a pagar (cuenta + propina)
- Calcular cuánto debe pagar cada persona

4. Presentación de resultados:

- Mostrar todos los resultados de forma clara y organizada
- Los montos deben mostrarse con exactamente 2 decimales
- Incluir el símbolo de moneda (\$) en los montos
- Usar f-strings para formatear los mensajes

3.3. Ejemplo de Ejecución

A continuación, se muestra un ejemplo de cómo debería funcionar tu programa:

Listing 1: Ejemplo de ejecución del programa

```
1  === CALCULADORA DE PROPINAS ===
2
3  Por favor, ingresa tu nombre: juan carlos
4  Hola, Juan Carlos! Bienvenido a la Calculadora de Propinas.
5
6  Ingresa el monto total de la cuenta: 150.50
7  Que porcentaje de propina deseas dejar? (ejemplo: 10, 15, 20): 15
8  Entre cuantas personas se dividira la cuenta? 4
9
10 --- RESUMEN DE LA CUENTA ---
11 Cuenta original: $150.50
12 Propina (15%): $22.57
13 Total a pagar: $173.07
14 Cada persona debe pagar: $43.27
15
16 Gracias por usar la Calculadora de Propinas!
```

4. Requisitos Técnicos

4.1. Estructura del Código

Tu programa debe incluir:

- **Comentarios:** Al inicio del archivo, incluye:
 - Tu nombre completo
 - Fecha
 - Descripción breve del programa
- **Variables con nombres descriptivos:** No uses nombres como `x`, `y`, `a`. Usa nombres claros como `nombre_usuario`, `monto_cuenta`, `porcentaje_propina`, etc.
- **Conversión de tipos:** Los datos ingresados con `input()` son strings. Debes convertirlos al tipo apropiado:
 - Usa `float()` para números decimales (montos, porcentajes)
 - Usa `int()` para números enteros (cantidad de personas)
- **Métodos de strings:** Usa al menos:
 - `.strip()` para eliminar espacios en blanco al inicio/final
 - `.title()` o `.capitalize()` para formatear el nombre del usuario
- **F-strings:** Todos los mensajes con variables deben usar f-strings. Ejemplo:

```
1 print(f"Total a pagar: ${total:.2f}")
```
- **Formato de números:** Los montos monetarios deben mostrarse con 2 decimales usando `:.2f`

4.2. Consejos y Recomendaciones

- **Planifica antes de programar:** Escribe en papel los pasos que debe seguir tu programa.
- **Prueba frecuentemente:** Ejecuta tu programa después de escribir cada parte para verificar que funciona.
- **Atención a los tipos de datos:** Recuerda que `input()` siempre devuelve un string. Si necesitas hacer matemáticas, debes convertir a número.
- **Cuidado con la división:** En Python 3, el operador `/` siempre produce un float. Esto es perfecto para esta tarea.
- **Formato de salida:** Haz que tu programa se vea profesional con líneas separadoras y mensajes claros.
- **Comentarios útiles:** Explica las partes complejas de tu código, especialmente los cálculos.

5. Entregables

Debes entregar:

1. Un archivo Python llamado `calculadora_propinas.py` con tu código completo y funcional.
2. El código debe ejecutarse sin errores.
3. Incluye comentarios que expliquen las secciones principales de tu código.

6. Defensa del Código (60 % de la nota final)

6.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

6.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando tu código en el editor.
2. **Ejecutar tu programa** en vivo y demostrar que funciona correctamente.
3. **Explicar tu código línea por línea:**
 - Por qué elegiste esos nombres de variables
 - Cómo funcionan las conversiones de tipo que usaste
 - Qué hacen los métodos de strings que aplicaste
 - Cómo realizaste los cálculos matemáticos
 - Cómo formateaste la salida con f-strings
4. **Responder preguntas** sobre tu implementación, por ejemplo:
 - “¿Qué pasa si cambias este número por otro?”
 - “¿Por qué usaste `float()` aquí y no `int()`?”
 - “¿Qué hace el método `.strip()` en esta línea?”
 - “¿Cómo modificarías tu código para hacer X?”
5. **Demostrar comprensión:** Debes poder explicar con tus propias palabras qué hace cada parte de tu código.

6.3. ¿Por qué es necesaria la defensa?

La defensa nos permite verificar que:

- Realmente entiendes el código que escribiste
- No copiaste código sin comprender cómo funciona
- Puedes explicar y razonar sobre tus decisiones de programación
- Estás aprendiendo los conceptos, no solo entregando una tarea

6.4. Consejos para una buena defensa

- **Practica tu explicación:** Antes de la defensa, repasa tu código y practica explicarlo en voz alta.
- **Entiende cada línea:** No dejes ninguna línea sin comprender. Si usaste algo, debes saber por qué.
- **Prepara tu entorno:** Asegúrate de que tu programa ejecuta correctamente antes de la reunión.
- **Sé honesto:** Si usaste ayuda (tutoriales, compañeros, etc.), menciónalo. Lo importante es que entiendas el resultado final.
- **Ten ejemplos listos:** Prepara diferentes valores de entrada para probar tu programa durante la defensa.

6.5. Distribución de la nota

- **Código funcional (40 %):** El programa funciona según los requisitos
- **Defensa del código (60 %):** Tu capacidad de explicar y defender tu trabajo

Importante: Si no asistes a la defensa o no puedes explicar tu código, la nota máxima de la tarea será el 40 %, sin importar qué tan bien funcione tu programa.

7. Desafíos Opcionales (Puntos Extra)

Si terminas la tarea principal y quieres seguir aprendiendo, intenta estos desafíos:

1. **Validación básica:** Agrega mensajes si el usuario ingresa 0 personas (no se puede dividir entre 0).
2. **Múltiples propinas:** Calcula y muestra cuánto sería con 10 %, 15 % y 20 % de propina al mismo tiempo.
3. **Redondeo inteligente:** Además de mostrar el monto exacto, muestra una versión redondeada al peso/dólar más cercano.
4. **Personalización:** Permite al usuario elegir el símbolo de moneda (\$, €, etc.).

Nota: Los desafíos opcionales suman puntos extra, pero primero asegúrate de completar correctamente los requisitos principales.

8. Rúbrica de Evaluación

8.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +15 puntos adicionales

8.2. Parte 1: Código Funcional (40 puntos)

8.2.1. 1.1 Funcionalidad Básica (20 puntos)

- **Ejecución sin errores (8 pts):**
 - 8 pts: Programa ejecuta perfectamente sin errores
 - 6 pts: Ejecuta con 1 error menor que no afecta el resultado final
 - 3 pts: Ejecuta con errores que afectan parcialmente los resultados
 - 0 pts: No ejecuta o crashea constantemente
- **Entrada de datos (6 pts):**
 - 6 pts: Solicita todos los datos (nombre, cuenta, propina, personas) con mensajes claros
 - 4 pts: Solicita todos los datos pero mensajes poco claros
 - 2 pts: Falta algún dato requerido
 - 0 pts: No solicita datos apropiadamente
- **Cálculos correctos (6 pts):**
 - 6 pts: Todos los cálculos (propina, total, división) son correctos
 - 4 pts: Un cálculo incorrecto
 - 2 pts: Dos cálculos incorrectos
 - 0 pts: Tres o más cálculos incorrectos o no realiza cálculos

8.2.2. 1.2 Uso de Conceptos de Python (12 puntos)

- **Variables descriptivas (3 pts):**
 - 3 pts: Nombres claros y descriptivos en todas las variables
 - 2 pts: Mayoría de nombres aceptables, algunos confusos
 - 1 pt: Nombres poco claros pero funcionales
 - 0 pts: Nombres inapropiados (x, y, a, var1, etc.)
- **Conversión de tipos (3 pts):**
 - 3 pts: Usa `float()` e `int()` correctamente en todos los casos necesarios
 - 2 pts: Usa conversiones correctamente pero falta algún caso
 - 1 pt: Usa conversiones pero con errores

- 0 pts: No usa conversión de tipos o completamente incorrecto

■ **F-strings (3 pts):**

- 3 pts: Usa f-strings en todos los mensajes con variables
- 2 pts: Usa f-strings en la mayoría de mensajes
- 1 pt: Usa f-strings solo en algunos casos
- 0 pts: No usa f-strings

■ **Métodos de strings (3 pts):**

- 3 pts: Usa `.strip()` y `.title()` o `.capitalize()` correctamente
- 2 pts: Usa solo uno de los métodos correctamente
- 1 pt: Intenta usar pero con errores
- 0 pts: No usa métodos de strings

8.2.3. 1.3 Formato y Presentación (8 puntos)

■ **Formato de montos (3 pts):**

- 3 pts: Todos los montos con exactamente 2 decimales y símbolo \$
- 2 pts: Formato correcto pero inconsistente
- 1 pt: Formato parcial o sin símbolo de moneda
- 0 pts: Sin formato apropiado

■ **Saludo personalizado (2 pts):**

- 2 pts: Nombre formateado correctamente con mayúsculas apropiadas
- 1 pt: Muestra nombre pero sin formato
- 0 pts: No personaliza el saludo

■ **Presentación clara (3 pts):**

- 3 pts: Salida bien organizada, profesional y fácil de leer
- 2 pts: Salida aceptable pero mejorable
- 1 pt: Salida desorganizada
- 0 pts: Salida confusa o difícil de entender

8.3. Parte 2: Defensa del Código (60 puntos)

8.3.1. 2.1 Comprensión del Código (30 puntos)

■ **Explicación de variables (8 pts):**

- 8 pts: Explica claramente el propósito de cada variable y por qué eligió esos nombres
- 6 pts: Explica la mayoría de variables con claridad
- 3 pts: Explicación superficial o confusa
- 0 pts: No puede explicar sus variables

■ Explicación de conversiones de tipo (8 pts):

- 8 pts: Explica perfectamente por qué usa `float()`, `int()` y cuándo usar cada uno
- 6 pts: Explica correctamente pero con alguna duda
- 3 pts: Explicación parcial o incorrecta
- 0 pts: No entiende las conversiones de tipo

■ Explicación de cálculos (8 pts):

- 8 pts: Explica clara y correctamente cada operación aritmética
- 6 pts: Explica correctamente con pequeñas imprecisiones
- 3 pts: Explicación confusa o parcialmente incorrecta
- 0 pts: No puede explicar los cálculos

■ Explicación de métodos y formato (6 pts):

- 6 pts: Explica perfectamente el uso de métodos de strings y f-strings
- 4 pts: Explica correctamente con alguna imprecisión
- 2 pts: Explicación superficial
- 0 pts: No entiende los métodos que usó

8.3.2. 2.2 Demostración y Modificación (20 puntos)**■ Ejecución en vivo (8 pts):**

- 8 pts: Ejecuta el programa correctamente con diferentes valores de entrada
- 6 pts: Ejecuta con algunos problemas menores
- 3 pts: Dificultades para ejecutar su propio programa
- 0 pts: No puede ejecutar el programa

■ Respuesta a preguntas técnicas (12 pts):

- 12 pts: Responde correcta y claramente a todas las preguntas sobre su código
- 9 pts: Responde bien la mayoría de preguntas, algunas dudas
- 5 pts: Respuestas parciales o confusas
- 2 pts: Dificultad para responder preguntas básicas
- 0 pts: No puede responder preguntas sobre su código

8.3.3. 2.3 Documentación y Buenas Prácticas (10 puntos)**■ Comentarios en el código (5 pts):**

- 5 pts: Comentarios claros al inicio y en secciones clave del código
- 3 pts: Comentarios presentes pero básicos o incompletos
- 1 pt: Comentarios mínimos
- 0 pts: Sin comentarios

■ Legibilidad del código (5 pts):

- 5 pts: Código perfectamente indentado, espaciado apropiado, muy legible
- 3 pts: Código legible con errores menores de formato
- 1 pt: Código funcional pero poco legible
- 0 pts: Código difícil de leer o entender

8.4. Puntos Extra (Hasta +15 puntos)

- **Desafío 1 - Validación básica (+4 pts):** Incluye validación para evitar división entre cero u otros errores lógicos
- **Desafío 2 - Múltiples propinas (+5 pts):** Calcula y muestra resultados con diferentes porcentajes de propina (10 %, 15 %, 20 %)
- **Desafío 3 - Redondeo inteligente (+3 pts):** Muestra versión redondeada al peso/dólar más cercano además del monto exacto
- **Desafío 4 - Personalización de moneda (+3 pts):** Permite al usuario elegir el símbolo de moneda

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicar cómo funcionan durante la defensa.

8.5. Penalizaciones

- **-100 % de la defensa:** No asiste a la defensa sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Asiste pero evidencia clara de no entender el código (posible plagio)
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-3 pts:** Nombre de archivo incorrecto
- **-2 pts:** Falta el encabezado con nombre, fecha y descripción

8.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

8.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de los conceptos
- **80-89 pts:** Muy Bueno - Comprensión sólida con errores menores
- **70-79 pts:** Bueno - Comprensión aceptable, necesita refuerzo
- **60-69 pts:** Suficiente - Comprensión básica, necesita mejorar
- **0-59 pts:** Insuficiente - No aprobado

9. Recursos de Apoyo

- **Documentación oficial de Python:** <https://docs.python.org/es/3/>
- **Funciones básicas:** Revisa las notas de clase sobre `print()`, `input()`, y type casting
- **F-strings:** Busca ejemplos de formateo de strings en Python 3.6+
- **Métodos de strings:** Experimenta en el intérprete interactivo de Python con `.strip()`, `.title()`, etc.

10. Preguntas Frecuentes

P: ¿Qué hago si mi programa da error?

R: Lee cuidadosamente el mensaje de error. Python te dice en qué línea está el problema y qué tipo de error es. Busca esa línea y verifica la sintaxis.

P: ¿Cómo muestro un número con 2 decimales?

R: Usa f-strings con el formato `:.2f`. Ejemplo: `print(f"{valor:.2f}")`

P: ¿Por qué obtengo error al sumar?

R: Probablemente intentas sumar strings. Recuerda convertir con `float()` o `int()` primero.

P: ¿Puedo usar otra estructura de mensajes?

R: Sí, siempre que cumplas todos los requisitos funcionales y técnicos.

P: ¿Es obligatorio hacer los desafíos opcionales?

R: No, son opcionales y solo dan puntos extra. Enfócate primero en completar bien los requisitos principales.

¡Éxito en tu primera tarea de programación!

Recuerda: todo programador experto fue alguna vez principiante.

La práctica constante es la clave del aprendizaje.